

Dualsub Project Plan Spec Kit Base

DualSub Learning Engine - Documento base de proyecto

0. Resumen ejecutivo

DualSub Learning Engine es una familia de aplicaciones y módulos orientados a aprendizaje de idiomas mediante reproducción de video/audio, transcripción, traducción y doble subtítulo sincronizado.

La idea no debe arrancar como "una app gigante", sino como un motor reutilizable que pueda conectarse a diferentes fuentes de contenido:

1. Videos o audios locales de la galería.
2. Modo de transcripción completa sin video.
3. YouTube embebido dentro de la app.
4. App reconocida por Android como reproductor externo.
5. Procesamiento offline para hacer compatible contenido con doble subtítulo.
6. Experimentos futuros de subtítulo en tiempo real.

El objetivo inicial recomendado es construir primero DualSub Local, una app Android en Flutter que permita importar un video/audio local, transcribirlo, traducirlo y reproducirlo con doble subtítulo sincronizado. Esta base técnica permite validar el núcleo del producto sin depender todavía de YouTube, DRM, streaming externo ni integración profunda con otras apps.

1. Objetivo del producto

Crear una herramienta privada/inicialmente personal para facilitar el aprendizaje de idiomas, especialmente inglés, mediante contenido real que el usuario ya consume.

La app debe permitir:

- Ver videos con subtítulos dobles.
- Escuchar audios con transcripción sincronizada.
- Traducir frases de forma contextual.
- Repetir frases para shadowing.
- Guardar vocabulario o frases útiles.
- Exportar subtítulos o transcripciones.
- Eventualmente abrir contenido desde otras apps Android.

Usuario objetivo inicial

Persona que consume videos, series, películas, YouTube o audios en una tablet/teléfono Android y quiere aprender inglés de forma sencilla, visual y contextual.

Principio rector

La app no debe sentirse como una herramienta técnica de subtítulos, sino como un reproductor de aprendizaje.

2. Nombre provisional

Opciones:

- DualSub
- LinguaMirror
- SubStudy
- EchoSubs
- FluentLayer

Para efectos técnicos del documento se usará:

DualSub Learning Engine

3. Problemas que resuelve

Problema 1: El usuario ve contenido pero no entiende todo

Muchas personas aprenden mejor con contenido real, pero pierden contexto si solo tienen subtítulos en inglés o solo traducción.

Problema 2: Las apps existentes son fragmentadas

Algunas apps sirven para YouTube, otras para archivos, otras para transcribir, otras para estudiar vocabulario. El objetivo es integrar gradualmente esas capacidades.

Problema 3: Android/tablet es el entorno prioritario

El uso principal será en Android, posiblemente tablet. La experiencia debe diseñarse desde el inicio para vertical y horizontal.

Problema 4: No todo contenido trae subtítulos

Por eso el sistema debe poder generar transcripción y traducción cuando el contenido no tiene subtítulos nativos.

4. Alcance general

En alcance

- App Android desarrollada principalmente en Flutter.
- Reproductor de video/audio local.
- Doble subtítulo sincronizado.
- Transcripción automática de audio.
- Traducción automática de transcripción.
- Edición manual básica de segmentos.
- Exportación a SRT/VTT/JSON.

- Modo estudio con repetición por frase.
- Integración futura con YouTube embebido.
- Integración futura como reproductor externo Android.

Fuera de alcance inicial

- Descargar videos de YouTube.
- Romper DRM o capturar contenido protegido.
- Subtitulado perfecto en tiempo real para cualquier app externa.
- Traducción profesional humana.
- Sincronización exacta palabra por palabra en el MVP.
- Soporte completo iOS/web/desktop desde la primera versión.

5. División del proyecto en productos/módulos

5.1 DualSub Core

Motor común del proyecto.

Responsabilidades:

- Modelo de datos de subtítulos.
- Timeline de reproducción.
- Sincronización por timestamp.
- Offset manual.
- Renderizado de subtítulos dobles.
- Exportación/importación SRT, VTT y JSON.
- Servicios abstractos de transcripción y traducción.
- Estados de proyecto: pendiente, procesando, listo, error.

Este módulo debe ser reusable por todas las apps.

5.2 DualSub Local

Primera app recomendada.

Permite:

- Importar video/audio local.
- Extraer audio.
- Transcribir.
- Traducir.
- Crear segmentos sincronizados.
- Reproducir con doble subtítulo.
- Guardar proyecto localmente.

Esta es la base del MVP.

5.3 DualSub Transcript

Modo sin video.

Permite:

- Cargar audio o video.
- Ver transcripción completa por bloques.
- Leer original + traducción.
- Reproducir frase por frase.
- Exportar texto, SRT, VTT, CSV o Anki.

Puede vivir dentro de DualSub Local como una pestaña o modo alternativo.

5.4 DualSub YouTube

Módulo para ver YouTube dentro de la app.

Permite:

- Pegar link de YouTube.
- Reproducir video embebido.
- Mostrar panel de subtítulos doble debajo del video.

- Sincronizar subtítulos propios con el tiempo de reproducción.
- Cargar subtítulos externos.
- Usar transcripción generada si el usuario provee audio o fuente compatible.

Restricción importante:

- No basar el MVP en descargar videos de YouTube.
- No depender inicialmente de extraer captions oficiales automáticamente.

5.5 DualSub External Player

App reconocida por Android como reproductor externo.

Permite:

- Aparecer en "Abrir con..." cuando otra app comparta un video compatible.
- Recibir URLs tipo content://, file:// o https://.
- Reproducir el contenido si el stream es accesible.
- Detectar o cargar subtítulos externos.
- Procesar contenido local cuando sea posible.

Requiere integración nativa Android mediante Kotlin/Java y configuración de AndroidManifest.xml .

5.6 DualSub Streaming Assist

Módulo experimental futuro.

Objetivo:

-Probar subtitulado en tiempo real sobre streams externos.

Estado:

- No recomendado para MVP.
- Alto riesgo por latencia, permisos, DRM, batería, sincronización y acceso al audio.

6. Matriz de viabilidad

Módulo	Viabilidad	Complejidad	Prioridad	Comentario
DualSub Core	Muy alta	Media	Crítica	Base de todo el sistema
DualSub Local	Muy alta	Media	Crítica	Mejor MVP
DualSub Transcript	Muy alta	Media	Alta	Puede integrarse al MVP

Módulo	Viabilidad	Complejidad	Prioridad	Comentario
DualSub YouTube	Media/ alta	Alta	Media	Mejor después del reproductor local
External Player Android	Media	Alta	Media/ baja	Requiere nativo Android
Streaming tiempo real	Baja/ media	Muy alta	Baja	Fase experimental
Procesamiento offline para doble subtítulo	Alta	Media/alta	Alta	Mejor que tiempo real para muchos casos

7. Stack técnico recomendado

7.1 Frontend / App

Flutter + Dart

Uso principal:

- Interfaz Android.
- Pantallas vertical/horizontal.
- Timeline de subtítulos.
- Overlay de doble subtítulo.
- Modo estudio.
- Historial de proyectos.
- Configuración de idioma.

- Edición manual de segmentos.

Flutter se recomienda porque permite una UI moderna, rápida de iterar y eventualmente portable a otras plataformas.

7.2 Capa nativa Android

Kotlin / Java

Uso necesario para:

- Recibir intents como reproductor externo.
- Manejar URLs Android complejas.
- Integración profunda con Media3/ExoPlayer si se decide usar reproductor nativo.
- Permisos de almacenamiento.
- Comunicación con Flutter mediante Platform Channels o Pigeon.

7.3 Reproductor multimedia

Opciones:

Opción A - Flutter-first

- media_kit
- Overlay de subtítulos propio en Flutter

Ventajas:

- Más control visual.
- Menos dependencia de UI nativa.
- Buena ruta para MVP.

Riesgos:

- Integración Android avanzada puede requerir trabajo extra.

Opción B - Android Media3 / ExoPlayer

Ventajas:

- Stack moderno y robusto para Android.
- Buen soporte de formatos.
- Mejor para app reconocida como reproductor externo.
- Mejor manejo de streams.

Riesgos:

- Más código nativo.
- Requiere puente con Flutter.

Recomendación

MVP:

Flutter + reproductor compatible + overlay custom de subtítulos.

Fase avanzada:

Integrar Media3/ExoPlayer para reproducción externa y streams más complejos.

7.4 Procesamiento de audio/video

FFmpeg / FFmpeg Kit

Uso:

- Extraer audio desde video.
- Convertir audio a formato compatible con ASR.
- Cortar segmentos.
- Normalizar audio.
- Exportar clips de frases.
- Generar archivos intermedios.

Ejemplo conceptual:

```
ffmpeg -i input.mp4 -vn -acodec pcm_s16le -ar 16000 -ac 1 output.wav
```

En app Flutter puede usarse un wrapper compatible, pero se debe revisar licencia, mantenimiento y tamaño final de APK.

7.5 Backend

Firestore recomendado para MVP extendido

Servicios:

- Firebase Auth: usuarios.
- Firestore: proyectos, metadatos, progreso.
- Firebase Storage: media, audio procesado, subtítulos.
- Cloud Functions o Cloud Run: procesamiento pesado.
- Firebase Emulator Suite: pruebas locales.

Decisión importante

Para uso privado/local, se puede iniciar sin login y guardar proyectos localmente.

Pero si se quiere calidad, sincronización y escalabilidad:

Firebase + backend de procesamiento es más sano que intentar hacer todo en el dispositivo.

7.6 Transcripción automática / ASR

Opciones posibles:

OpenAI Audio Transcriptions

Uso:

- Transcribir audio.
- Obtener texto.
- Obtener segmentos/timestamps cuando el formato/modelo lo permita.

- Eventualmente diarización si se necesita separar voces.

Ventajas:

- Alta calidad.
- Buena integración con flujos de IA.
- Útil para inglés/español y otros idiomas.

Riesgos:

- Costo por uso.
- Dependencia de API externa.
- Requiere backend seguro para no exponer API keys.

Google Cloud Speech-to-Text

Uso:

- Transcripción robusta.
- Posible soporte de muchos idiomas.
- Integración natural si se usa Firebase/Google Cloud.

Ventajas:

- Ecosistema Google.
- Escalable.
- Integración con Cloud Storage.

Riesgos:

- Configuración más empresarial.
- Costos y cuotas.

Whisper local / modelo local futuro

Uso:

- Transcripción en local o backend propio.

Ventajas:

- Más privacidad.
- Menos dependencia de APIs externas.
- Posible uso offline.

Riesgos:

- Más pesado.
- Móvil Android puede no ser suficiente para buen rendimiento.
- Requiere optimización o servidor propio.

7.7 Traducción

Opciones:

LLM vía API

Uso:

- Traducción contextual frase por frase.
- Corrección de estilo.
- Explicación de expresiones idiomáticas.
- Generación de notas de aprendizaje.

Ventajas:

- Mejor contexto lingüístico.
-Puede explicar frases, no solo traducir.

Riesgos:

- Costo.
- Latencia.
- Puede variar el estilo si no se controla con prompts.

Google Cloud Translation

Uso:

- Traducción más tradicional y rápida.
- Procesamiento por lotes.

Ventajas:

- Estable.
- Rápida.
- Buena para MVP de traducción básica.

Riesgos:

- Menos pedagógica/contextual.

7.8 Base de datos local

Opciones:

- SQLite / Drift.
- Isar.
- Hive.
- SharedPreferences solo para configuración simple.

Recomendación:

- Para MVP simple: SQLite/Drift o Isar.
- Para datos relacionales de proyectos, segmentos, vocabulario y progreso: Drift encaja bien.

8. Arquitectura general

8.1 Arquitectura lógica

```
Flutter App
  \vdash UI Layer
    Home
    Import Media
    Processing Status
    Player
    Transcript Mode
    Study Mode
```

```
Settings
\vdash Domain Layer
  \vdash MediaProject
    SubtitleSegment
    Transcript
```

```
VocabularyItem
StudySession
Services
  MediaImportService
  AudioExtractionService
  TranscriptionService
  TranslationService
  SubtitleSyncService
  ExportService
  StorageService
Infrastructure
  Local Database
  File System
  Firebase Client
  API Client
  Native Android Bridge
Native Android
  - Intent Receiver
  - Media3/ExoPlayer optional
\vdash URI Resolver
L Platform Channels
```

8.2 Pipeline MVP

1. Usuario importa video/audio
2. App crea MediaProject
3. App extrae audio
4. Audio se manda a ASR
5. ASR devuelve segmentos con timestamps
6. Backend/app traduce segmentos
7. App guarda DualSub JSON
8. Usuario reproduce contenido
9. Overlay muestra original + traducción según tiempo actual

10. Usuario ajusta offset si es necesario
11. Usuario exporta SRT/VTT/JSON si quiere

9. Modelo de datos inicial

9.1 MediaProject

```
{
  "id": "project_001",
  "title": "Example video",
  "mediaType": "video",
  "source": "local_file",
  "mediaUri": "content://media/video/123",
  "localMediaPath": "/app/documents/video.mp4",
  "audioPath": "/app/documents/audio.wav",
  "sourceLanguage": "en",
  "targetLanguage": "es",
  "status": "ready",
  "createdAt": "2026-05-01T12:00:00Z",
  "updatedAt": "2026-05-01T12:10:00Z"
}
```

9.2 SubtitleSegment

```
{
  "id": "seg_0001",
  "projectId": "project_001",
  "index": 1,
  "startMs": 1250,
  "endMs": 4800,
  "originalText": "I have been working all day.",
  "translatedText": "He estado trabajando todo el día.",
  "confidence": 0.91,
  "speaker": null,
  "isEdited": false,
  "notes": []
}
```

9.3 VocabularyItem

```
{
  "id": "vocab_001",
  "projectId": "project_001",
  "segmentId": "seg_0001",
  "term": "working all day",
```

```
  "translation": "trabajando todo el día",
  "note": "Useful phrase for daily routines",
  "createdAt": "2026-05-01T12:20:00Z"
}
```

9.4 StudySession

```
{
  "id": "study_001",
  "projectId": "project_001",
  "startedAt": "2026-05-01T12:30:00Z",
  "endedAt": "2026-05-01T12:50:00Z",
  "segmentsReviewed": 24,
  "phrasesSaved": 5,
  "shadowingLoops": 12
}
```

10. Formato propio DualSub JSON

El proyecto debe tener un formato interno propio, aunque también exporte a SRT/VTT.

```
{
  "schemaVersion": "1.0.0",
  "project": {
    "id": "project_001",
    "title": "Example video",
    "sourceLanguage": "en",
    "targetLanguage": "es",
    "durationMs": 120000
  },
  "segments": [
    {
      "id": "seg_0001",
      "startMs": 1250,
```



```
    "endMs": 4800,  
    "original": "I have been working all day.",  
    "translation": "He estado trabajando todo el día.",  
    "speaker": null,  
    "confidence": 0.91  
  }  
}
```

```
  ]  
}
```

Ventaja del formato propio:

- Permite más datos que SRT.
- Puede guardar vocabulario, notas, confianza, speaker y estado de edición.
- Se puede convertir a SRT/VTT después.

11. UI/UX requerida

11.1 Home

Debe mostrar:

- Botón "Importar video/audio".
- Botón "Pegar link de YouTube" en fases futuras.
- Lista de proyectos recientes.
- Estado de procesamiento.
- Acceso rápido a vocabulario guardado.

11.2 Import Media

Debe permitir:

- Seleccionar video.
- Seleccionar audio.
- Elegir idioma original.
- Elegir idioma destino.

- Elegir modo:
- Reproducir con subtítulos.
- Solo transcripción.
- Procesar para estudiar después.

11.3 Processing Screen

Debe mostrar:

- Archivo seleccionado.
- Pasos del pipeline:
- Extrayendo audio.
- Transcribiendo.
- Traduciendo.
- Generando subtítulos.
- Listo.
- Manejo claro de errores.
- Opción de reintentar.

11.4 Player vertical

Layout recomendado:

```
[ Video ]
[ Original subtitle ]
[ Translated subtitle ]
[ Controls: rewind phrase | loop | hide translation | save phrase ]
[ Transcript list / vocabulary ]
```

11.5 Player horizontal

Layout recomendado:

```
[ Full video ]
Overlay bottom:
```

Original subtitle
Translated subtitle

Controles rápidos:

- Tap para pausar.
- Swipe vertical para ocultar/mostrar traducción.
- Botón loop frase.
 - Botón guardar frase.

11.6 Transcript Mode

Debe mostrar:

[00:01.250-00:04.800]

I have been working all day.

He estado trabajando todo el día.

[Play phrase] [Loop] [Save] [Edit]

11.7 Editor básico de segmentos

MVP:

- Editar texto original.
- Editar traducción.
- Ajustar inicio/fin manualmente.
- Unir segmentos.
- Dividir segmento.

Fase posterior:

- Waveform visual.
- Edición palabra por palabra.
- Alineación automática fina.

12. Requisitos funcionales MVP

RF-001 Importar archivo local

Como usuario, quiero seleccionar un video o audio desde mi dispositivo para generar subtítulos dobles.

Criterios de aceptación:

- La app permite seleccionar un archivo local.
- La app detecta si es video o audio.
- La app crea un proyecto.
- La app muestra error si el formato no es compatible.

RF-002 Extraer audio

Como sistema, quiero extraer el audio de un video para enviarlo a transcripción.

Criterios de aceptación:

- De un MP4 compatible se genera un archivo de audio intermedio.
- El archivo intermedio puede enviarse al servicio de transcripción.
- Si falla, se muestra un error recuperable.

RF-003 Transcribir audio

Como usuario, quiero obtener la transcripción sincronizada del contenido.

Criterios de aceptación:

- El sistema envía audio al servicio ASR.
- El sistema recibe texto segmentado.
- Cada segmento tiene inicio y fin.
- La transcripción se guarda localmente.

RF-004 Traducir segmentos

Como usuario, quiero ver una traducción debajo de cada frase original.

Criterios de aceptación:

- Cada segmento original tiene traducción.
- La traducción conserva el orden de segmentos.
- La app permite reintentar traducción si falla.

RF-005 Reproducir con doble subtítulo

Como usuario, quiero ver el video/audio con subtítulos en dos idiomas.

Criterios de aceptación:

- El subtítulo mostrado corresponde al tiempo actual.
- Se muestran original y traducción.
- Hay modo vertical y horizontal.
- El usuario puede ocultar la traducción.

RF-006 Ajustar sincronización

Como usuario, quiero ajustar los subtítulos si aparecen antes o después.

Criterios de aceptación:

- Hay control de offset global.
- El offset modifica todos los segmentos visualmente.
- El offset puede guardarse en el proyecto.

RF-007 Modo frase/loop

Como usuario, quiero repetir una frase para practicar pronunciación.

Criterios de aceptación:

- La app permite saltar al inicio del segmento actual.
- La app permite repetir el segmento actual.
- El usuario puede salir del loop fácilmente.

RF-008 Exportar subtítulos

Como usuario, quiero exportar mis subtítulos.

Criterios de aceptación:

- La app exporta formato JSON propio.
- La app exporta SRT.
- La app exporta VTT.

13. Requisitos no funcionales

Rendimiento

- El player debe mantener reproducción fluida.
- El overlay no debe trabarse al actualizar subtítulos.
- El timeline debe buscar segmento activo de forma eficiente.

Privacidad

- No subir archivos sin confirmación clara.
- Avisar cuando se use API externa.
- No exponer API keys en la app móvil.
- Permitir borrar proyectos y archivos generados.

Calidad de transcripción

- Guardar nivel de confianza si la API lo proporciona.
- Permitir edición manual.
- Mostrar claramente cuando una transcripción fue generada automáticamente.

Robustez

- Manejar archivos grandes.
- Manejar errores de red.
- Manejar cancelación de procesamiento.

- Reintentar pasos fallidos sin reiniciar todo.

Observabilidad

- Registrar errores de procesamiento.
- Registrar duración de cada paso.
- Registrar tamaño de archivo y duración del audio.
- En modo desarrollo, mostrar logs técnicos.

14. APIs y servicios candidatos

14.1 OpenAI API

Uso potencial:

- Transcripción de audio.
- Traducción contextual.
- Explicación de frases.
- Generación de vocabulario.
- Resúmenes del contenido.
- Quiz automático.

Rutas de uso:

- Audio transcription para convertir audio a texto.
- Modelo de texto para traducción y explicación pedagógica.

Arquitectura segura:

```
Flutter App
  \downarrow
Backend propio / Cloud Function
  \downarrow
OpenAI API
```

No colocar API keys dentro de la app.

14.2 Google Cloud Speech-to-Text

Uso potencial:

- Transcripción principal o alternativa.
- Procesamiento batch.
- Integración con Firebase/Google Cloud.

14.3 Google Cloud Translation

Uso potencial:

- Traducción rápida y económica por lotes.
- Alternativa a traducción con LLM.

14.4 Firebase

Uso potencial:

- Auth.
- Firestore.
- Storage.
- Cloud Functions.
- Cloud Run.
- Emulator Suite.
- Crashlytics.
- Analytics.

14.5 YouTube IFrame Player API

Uso potencial:

- Reproducir YouTube embebido dentro de la app/webview.
- Controlar reproducción básica.
- Sincronizar subtítulos propios con el tiempo de reproducción.

Restricciones:

- Respetar términos de YouTube.
- No basar el producto en descarga de contenido.
- No asumir acceso libre a captions oficiales.

14.6 Android Media3 / ExoPlayer

Uso potencial:

- Reproducción nativa robusta en Android.
- Soporte de subtítulos externos.
- Reproducción de streams compatibles.
- Integración como reproductor externo.

14.7 FFmpeg

Uso potencial:

- Extracción de audio.
- Conversión de formatos.
- Cortes por segmento.
- Normalización.
- Exportaciones futuras.

Atención:

- Revisar licencias.
- Revisar tamaño de APK.
- Revisar compatibilidad Android.

15. Estrategia de desarrollo por fases

Fase 0 - Preparación técnica

Objetivo:

Validar que Flutter puede reproducir video/audio local y renderizar overlay sincronizado.

Tareas:

- Crear app Flutter base.
- Reproducir un video local.
- Crear lista mock de SubtitleSegment.
- Sincronizar subtítulo activo con posición del player.
- Probar vertical/horizontal.
- Probar offset global.

Entregable:

- Demo local sin backend ni ASR.

Fase 1 - MVP Local sin IA real

Objetivo:

Validar experiencia con archivos .srt , .vtt o JSON cargados manualmente.

Tareas:

- Importar archivo de video.
- Importar archivo de subtítulos.
- Parsear SRT/VTT.
- Mostrar doble subtítulo si el JSON ya contiene traducción.
- Exportar DualSub JSON.

Entregable:

- Reproductor funcional con subtítulos precargados.

Fase 2 - Transcripción automática

Objetivo:

Procesar audio/video local para generar subtítulos base.

Tareas:

- Integrar extracción de audio.
- Crear backend de transcripción.
- Enviar archivo al backend.
- Recibir segmentos.
- Guardar segmentos.
- Mostrar transcript.

Entregable:

- Video/audio local transcrito automáticamente.

Fase 3 - Traducción automática

Objetivo:

Completar doble subtulado generado.

Tareas:

- Crear servicio de traducción por segmentos.
- Mantener alineación 1:1 entre original y traducción.
- Implementar retries.
- Guardar traducciones.
- Mostrar original + traducción.

Entregable:

- Contenido local reproducible con doble subtítulo generado.

Fase 4 - Modo estudio

Objetivo:

Convertir la app en herramienta de aprendizaje, no solo reproducción.

Tareas:

- Repetir frase.

- Ocultar traducción.
- Guardar frase.
- Crear vocabulario.
- Exportar CSV/Anki.
- Modo shadowing básico.

Entregable:

- MVP útil para aprendizaje diario.

Fase 5 - YouTube embebido

Objetivo:

Permitir ver videos de YouTube dentro de la app con subtítulos propios.

Tareas:

- Pantalla de YouTube WebView/IFrame.
- Leer tiempo actual del player.
- Sincronizar overlay/panel de subtítulos.
- Cargar subtítulos externos o transcript generado.
- Permitir offset.

Entregable:

- YouTube + subtítulos propios.

Fase 6 - Reproductor externo Android

Objetivo:

Hacer que Android reconozca la app como reproductor de video.

Tareas:

- Configurar intent filters.
- Recibir ACTION_VIEW con MIME video/audio.
- Resolver URI.

- Abrir contenido en player.
- Manejar permisos temporales.
- Detectar subtítulos asociados si existen.

Entregable:

- App aparece como reproductor externo para contenido compatible.

Fase 7 - Procesamiento offline avanzado

Objetivo:

Cuando no sea viable subtitular en tiempo real, permitir procesar primero y estudiar después.

Tareas:

- Cola de procesamiento.
- Estados persistentes.
- Procesamiento en backend.
- Notificación cuando esté listo.
- Cache de resultados.

Entregable:

- "Preparar para doble subtitulado" en vez de depender de tiempo real.

16. Riesgos técnicos y mitigaciones

Riesgo: YouTube limita acceso a captions o descarga

Mitigación:

- Usar YouTube embebido.
- Usar subtítulos propios cargados/generados.
- No depender de descarga automática.

Riesgo: Streaming externo con DRM no permite acceso

Mitigación:

- Tratarlo como experimental.
- Priorizar archivos locales y URLs compatibles.
- Ofrecer flujo offline cuando el archivo sea accesible.

Riesgo: Latencia en transcripción/traducción

Mitigación:

- Procesamiento offline/asíncrono.
- Guardar resultados.
- Mostrar progreso claro.
- Reusar caché.

Riesgo: Traducciones desalineadas

Mitigación:

- Traducir por segmento, no texto completo sin estructura.
- Usar IDs de segmento.
- Validar que la respuesta conserve el número de segmentos.
- Reintentar segmentos fallidos individualmente.

Riesgo: App móvil lenta con archivos grandes

Mitigación:

- Usar backend para procesamiento pesado.
- Extraer audio comprimido/normalizado.
- Evitar cargar todo el transcript en memoria si es largo.

Riesgo: Exponer API keys

Mitigación:

- Nunca llamar APIs sensibles directamente desde Flutter.
- Usar backend propio o Cloud Functions.
- Aplicar autenticación y cuotas.

Riesgo: Complejidad excesiva

Mitigación:

- Empezar con DualSub Local.
- Mockear IA primero.
- Añadir YouTube y external player solo cuando Core esté estable.

17. Organización sugerida del repositorio

```
dualsub/  
  apps/  
    mobile_flutter/  
      lib/  
        core/  
        features/  
        player/  
        services/  
        data/  
        native_bridge/  
      android/  
      test/  
  backend/  
    functions/  
    api/  
    workers/  
    prompts/  
    tests/  
  packages/  
    dualsub_core/  
    subtitle_parser/  
    dualsub_models/  
  docs/
```

```
product/  
architecture/  
specs/  
api/  
ux/  
testing/  
scripts/  
  media_processing/  
  subtitle_conversion/
```

18. Estructura recomendada para Spec Kit

Este documento puede convertirse después en varios specs más pequeños.

18.1 Specs iniciales a generar

1. product-overview.md
2. technical-architecture.md
3. data-model.md
4. mvp-local-player.md
5. subtitle-sync-engine.md
6. transcription-pipeline.md
7. translation-pipeline.md
8. android-external-player.md
9. youtube-embedded-player.md
10. test-plan.md

18.2 Primer spec recomendado

El primer spec accionable debería ser:

MVP Local Player with Mock Dual Subtitles

No empezar con IA, YouTube ni reproductor externo.

Objetivo del primer spec

Crear una app Flutter que reproduzca un video local y muestre subtítulos dobles desde un JSON mock.

Resultado esperado

- Pantalla de importación.
- Pantalla de reproducción.
- Overlay doble.
- Timeline sincronizado.
- Offset global.
- Cambio vertical/horizontal.

19. Agentes recomendados para trabajar el proyecto

19.1 Director Agent

Responsabilidad:

- Mantener visión general.
- Revisar coherencia entre specs.
- Evitar sobreingeniería.
- Priorizar MVP.
- Validar decisiones técnicas.

Debe conocer:

- Este documento completo.
- Restricciones Android.
- Prioridad de Flutter.
- Objetivo de aprendizaje de idiomas.

19.2 Flutter Mobile Agent

Responsabilidad:

- UI Flutter.
- Estado de app.
- Player screen.
- Overlay de subtítulos.
- Navegación.
- Integración con servicios.

Debe conocer:

- Arquitectura Flutter.
- Diseño responsive vertical/horizontal.
- Manejo de estado.
- Testing de widgets.

19.3 Android Native Agent

Responsabilidad:

- AndroidManifest.
- Intent filters.
- Kotlin bridge.
- Media3/ExoPlayer si aplica.
- Permisos y URIs.

Debe conocer:

- Android intents.
- Storage Access Framework.
- Content URIs.
- Platform Channels/Pigeon.

19.4 Backend/AI Agent

Responsabilidad:

- APIs de transcripción.
- APIs de traducción.
- Cloud Functions/Cloud Run.
- Seguridad de API keys.
- Cola de procesamiento.
- Validación de respuestas.

Debe conocer:

- OpenAI/Google Cloud APIs.
- Firebase.
- Procesamiento de audio.
- Prompts estructurados.
- JSON schema validation.

19.5 Media Processing Agent

Responsabilidad:

- FFmpeg.
- Extracción de audio.
- Conversión de formatos.
- Exportación SRT/VTT.
- Normalización de audio.

Debe conocer:

- Contenedores multimedia.
- Codecs.
- Formatos de subtítulos.
- Limitaciones móviles.

19.6 QA/Test Agent

Responsabilidad:

- Casos de prueba.
- Validación de sincronización.
- Archivos edge case.
 - Pruebas Android reales.
- Regression testing.

Debe conocer:

- Testing Flutter.
- Testing Android.
- QA de reproductores.
- Casos de error de red/procesamiento.

19.7 UX/Learning Agent

Responsabilidad:

- Experiencia pedagógica.
- Shadowing.
- Vocabulario.
- Flujo para principiantes.
- Diseño para tablet.

Debe conocer:

- Aprendizaje de idiomas.
- UI simple.
- Diseño de interfaces de video.
- Fricción mínima para usuario no técnico.

20. Backlog inicial

-

Épica 1: Core Models

- Crear MediaProject .
- Crear SubtitleSegment .
- Crear DualSubTranscript .
- Crear parser de JSON.
- Crear exportador JSON.

Épica 2: Local Player

- Reproducir archivo local.
- Leer posición actual.
- Mostrar subtítulo activo.
- Mostrar original/traducción.
- Añadir offset.

Épica 3: Subtitle Import/Export

- Importar SRT.
- Importar VTT.
- Exportar SRT.
- Exportar VTT.
- Convertir a DualSub JSON.

Épica 4: Transcription Pipeline

- Extraer audio.
- Subir audio.
- Transcribir.
- Guardar segmentos.
- Mostrar pantalla de procesamiento.

Épica 5: Translation Pipeline

- Traducir segmentos.
- Validar correspondencia 1:1.
- Guardar traducción.
- Reintentar fallidos.

Épica 6: Study Mode

- Loop de frase.
- Ocultar traducción.
- Guardar frase.
- Crear vocabulario.
- Exportar vocabulario.

Épica 7: YouTube Embedded

- Pegar link.
- Reproducir en WebView/IFrame.
- Obtener tiempo actual.
- Sincronizar subtítulos propios.

Épica 8: External Player

- Registrar intent filter.
- Recibir URI.
- Resolver URI.
- Reproducir contenido.
- Manejar errores de acceso.

21. Definición de Done para MVP 0

MVP 0 queda completo cuando:

- La app abre en Android.
- Se puede cargar un video local de prueba.
- El video se reproduce.
- La app carga subtítulos mock desde JSON.
- Los subtítulos aparecen sincronizados.
- Se muestran dos líneas: original y traducción.
- El usuario puede ajustar offset global.
- En vertical los subtítulos aparecen debajo del video.
- En horizontal aparecen como overlay.
- Existe al menos una prueba automatizada para el selector de segmento activo.

22. Decisiones técnicas iniciales

Decisión 1

Empezar con Flutter para la app principal.

Decisión 2

No empezar con YouTube ni streaming externo.

Decisión 3

Construir primero el motor de subtítulos con datos mock.

Decisión 4

Mantener la IA detrás de un backend, no embebida directamente en Flutter.

Decisión 5

Usar formato propio JSON como fuente interna de verdad.

Decisión 6

Exportar SRT/VTT como compatibilidad, no como formato interno principal.

Decisión 7

Procesar offline/asíncrono antes de intentar tiempo real.

23. Preguntas pendientes para cerrar en siguiente iteración

Estas preguntas no bloquean el MVP 0, pero sí ayudan para las fases siguientes.

1. ¿El idioma principal será inglés → español al inicio?
2. ¿La app será solo privada para uso personal o eventualmente pública?
3. ¿Se prioriza tablet sobre teléfono?
4. ¿Se quiere login desde el inicio o modo local primero?
5. ¿Se aceptan APIs pagadas desde el MVP?
6. ¿Se prefiere OpenAI, Google Cloud o ambos como proveedores de transcripción?
7. ¿Se quiere exportar a Anki desde fase temprana?
8. ¿Se necesita soporte para japonés/francés desde temprano o después?
9. ¿El diseño visual debe parecer más a Language Reactor, SoundTypeAI o una app propia más moderna?
10. ¿Se quiere guardar progreso de estudio por usuario?

24. Siguiendo paso recomendado

Crear el primer documento formal de Spec Kit:

Spec 001 - DualSub Local Player with Mock Subtitles

Este spec debe convertir el MVP 0 en tareas técnicas concretas para un agente de desarrollo.

La implementación debe evitar IA al inicio. Primero se prueba la experiencia central:

```
video local + JSON mock + doble subtítulo sincronizado
```

Cuando eso funcione, el proyecto tendrá una base sólida para agregar transcripción, traducción, YouTube y reproductor externo sin rehacer la arquitectura.